



School of Computer Science

COMP 8347 “Internet Applications and Distributed
Systems”

Summer 2026

Week 9 - II

Message Queues & Microservices (Celery, RabbitMQ, Kafka)

DR. OLENA SYROTKINA



University of Windsor

Copyright Notice

This content is protected and may not be shared, uploaded, or distributed. Any unauthorized distribution, reproduction or sharing of this material, including uploading to **CourseHero**, **Chegg**, **StuDocu** or other websites, is **strictly prohibited** and may be subject to legal action. Students are granted access to this material solely for their personal use and may not use it for any other purpose without the express written consent of the instructor. Thank you for respecting our intellectual property rights.

Agenda

- Why Queues?
- What Are Microservices?
- Why Do Developers Use Microservices?
- Project: “Sticker Shop” (tiny e-commerce)

Why Queues?

Problem: Web apps often do too much work during the HTTP request.

Example:

User uploads a photo → server:

- Resizes image
- Scans for NSFW (Not Safe For Work) content
- Generates thumbnails
- Stores metadata
- Updates search index
- Sends notifications

This can take **seconds**, making the request slow, timeout, or fail.

Key point:

Your web server should handle **requests**, not **heavy jobs**.

Symptoms of Tight Coupling

Symptom	What it Looks Like	Why it Happens
Slow web responses	User hits submit → spinner → timeout	Web server blocks while doing heavy work
Cascading failures	One service becomes slow → everything becomes slow	Dependencies are directly chained together
Noisy neighbors	One large job slows down everyone else	All work is on the same machine/process

Why Queues?

Queues Introduce an Async Boundary

Instead of the web request doing the slow work, the web server **delegates**:

User → Django (API) → Queue → Worker(s) → External Tasks

1. User sends request
 2. Django writes a **message to a queue** (quick!)
 3. Returns response like: “Request accepted”
 4. Background **workers** pull messages from the queue
 5. Workers perform slow work **asynchronously**
- The user is **no longer waiting** on long tasks.

Benefits of Queues

Benefit	Explanation	Example
Latency Shielding	The user gets a response immediately	“We received your file. You will get an email when it is ready.”
Smoothing Spikes (Load Leveling)	Queue absorbs bursts; workers process steadily	10,000 uploads in 1 minute? Fine.
Isolation	Heavy tasks don’t slow down web requests	Image processing no longer freezes login page
Retryable Errors	If external service fails, workers retry	Payment API down for 5s? No user pain.
Horizontal Scaling	Need more speed? Add more workers, not web servers	Scale consumers independently

Why Queues?

Costs (Trade-offs)

Cost	Description
Eventual Consistency	The user's action doesn't complete immediately
Operational Complexity	Need to deploy / monitor queue + workers
Message Ordering Caveats	Messages may not be processed strictly in timestamp order

Important!

Queues improve performance by accepting that **some results arrive later**.

Where in Your Django App?

Browser → Django (View/API)
→ send_task()
→ Queue (Redis / RabbitMQ / SQS)
→ Worker(s) (Celery / RQ / Dramatiq)
→ DB / Storage / External APIs
→ Sends notifications or write results back

Why Queues?

Real Scenarios Where Queues Help

Scenario	Synchronous = Bad	With Queue = Good
Sending emails	Slow SMTP servers freeze the request	Worker sends email later
Generating PDFs	CPU spike blocks all requests	Worker runs CPU tasks separately
Video/Audio processing	Minutes of encoding	Process offline
Web scraping / API polling	External APIs can be slow	Worker retries safely

What Are Microservices?

A microservice is:

- **Small**
- **Focused on doing one thing**
- **Runs independently**
- Communicates with others using **APIs** (often HTTP / REST / gRPC / messaging)

For example, a shopping website might be split like this:

Service Name	Responsibility
User Service	login, accounts
Product Service	product catalog
Order Service	checkout, receipts
Payment Service	credit card processing
Email Service	sending order confirmations

Each service is deployed separately and can scale separately.

Why Do Developers Use Microservices?

Benefits:

- **Scalable:** You can scale *just* the busy part (e.g., Product Service) instead of the whole system.
- **Faster development:** Teams work independently without stepping on each other's code.
- **Resilient:** If one service crashes, others can still run.
- **Easy to update:** You can upgrade one service without redeploying everything.

Downsides:

- **More complexity:** Many small services means networking, monitoring, logging, versioning, etc.
- **More ops overhead:** Need DevOps tooling (Kubernetes, Docker, service registry, tracing).
- **Harder debugging:** A bug might involve multiple services talking to each other.

Project: “Sticker Shop” (tiny e-commerce)

What it does

- Users place an order for stickers via a REST API.
- The API **enqueues** background work (send confirmation email, generate invoice PDF).
- Workers process those jobs **asynchronously** via **Celery + RabbitMQ**.
- Each important event (order_created, invoice_generated, email_sent) is **published to Kafka**.
- A separate **Analytics microservice** consumes Kafka events and prints/stores simple metrics.

Celery

Celery is a **task queue** for Python. It allows you to run **work in the background**, outside the main web request.

You use it when something is **too slow to do during the request**, like sending emails, generating reports, processing images, etc.

Key idea:

Your web app says “*Do this task later*”, and Celery runs it in a worker process.

RabbitMQ

RabbitMQ is a **message broker**.

It sits between producers and consumers and ensures messages are **delivered reliably**.

Key idea:

Applications send messages to RabbitMQ → RabbitMQ stores and routes them → Workers receive and process them.

Often used **with Celery** as its message transport.

Kafka

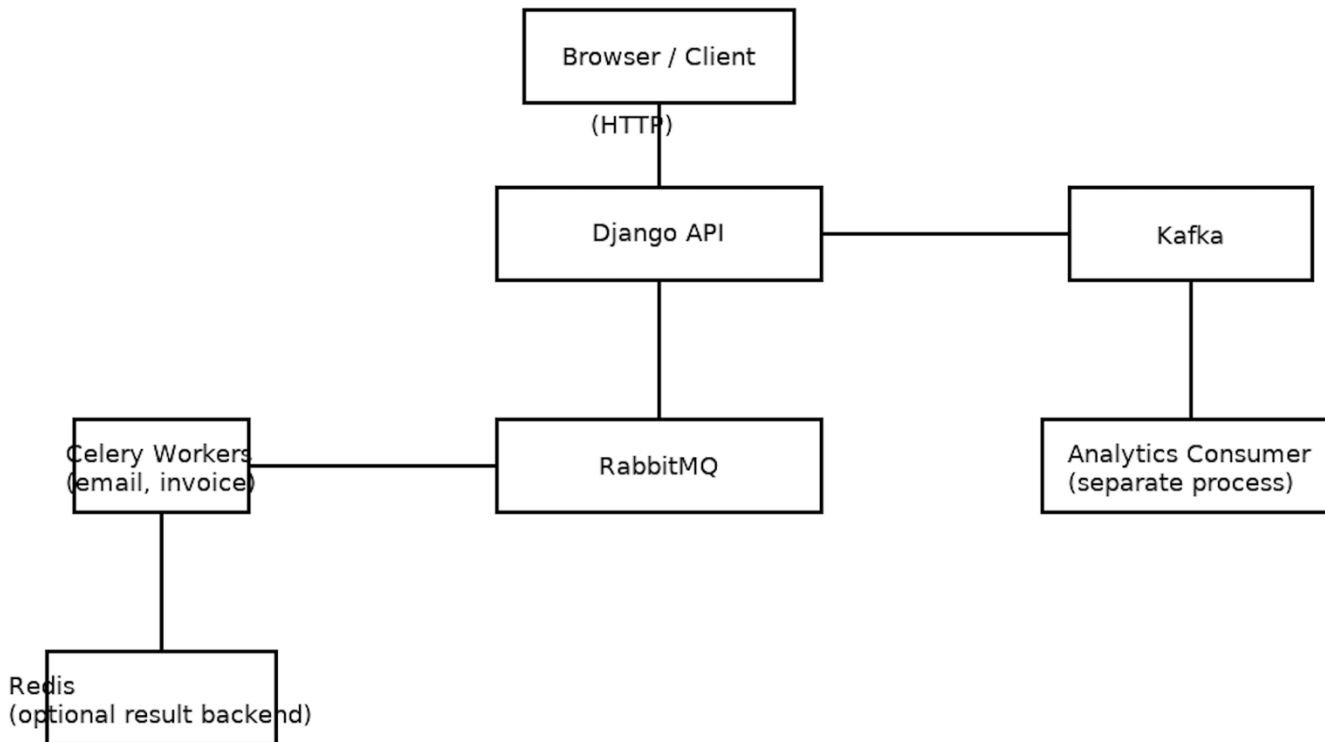
Kafka is a **distributed event streaming platform**.

Instead of sending one message to one worker, Kafka stores **streams of events** that many services can read, replay, and process.

Key idea:

Kafka is built for **high-throughput, real-time data pipelines**, logs, metrics, analytics, and event-driven architectures.

Project: “Sticker Shop” – Big picture



- **Django API** = synchronous request/response. It never “does the heavy work”; it just validates input, writes the order to DB, and enqueues tasks.
- **RabbitMQ** = message broker for Celery tasks (reliable queue).
- **Celery Workers** = pick up tasks (send email, generate invoice).
- **Kafka** = event stream (append-only log). We push events to Kafka so **other services** (like Analytics) can react **without coupling** to our API.
- **Analytics Consumer** = reads events from Kafka and aggregates/prints stats.

Project: “Sticker Shop”

Step 1 – Create a Folder for the Project

Open Terminal and run:

```
mkdir sticker-shop  
cd sticker-shop
```

This folder will hold **everything**.

Step 2 – Create a Python Virtual Environment

We always use virtual environments to keep dependencies clean.

```
#MacOS  
python3 -m venv venv  
source venv/bin/activate
```

```
#Windows  
python -m venv venv  
venv\Scripts\activate
```

You should now see **(venv)** in your terminal prompt.

Project: “Sticker Shop”

Step 3 – Install Django, DRF, Celery, etc.

#MacOS

```
pip3 install django djangorestframework celery redis
```

#Windows

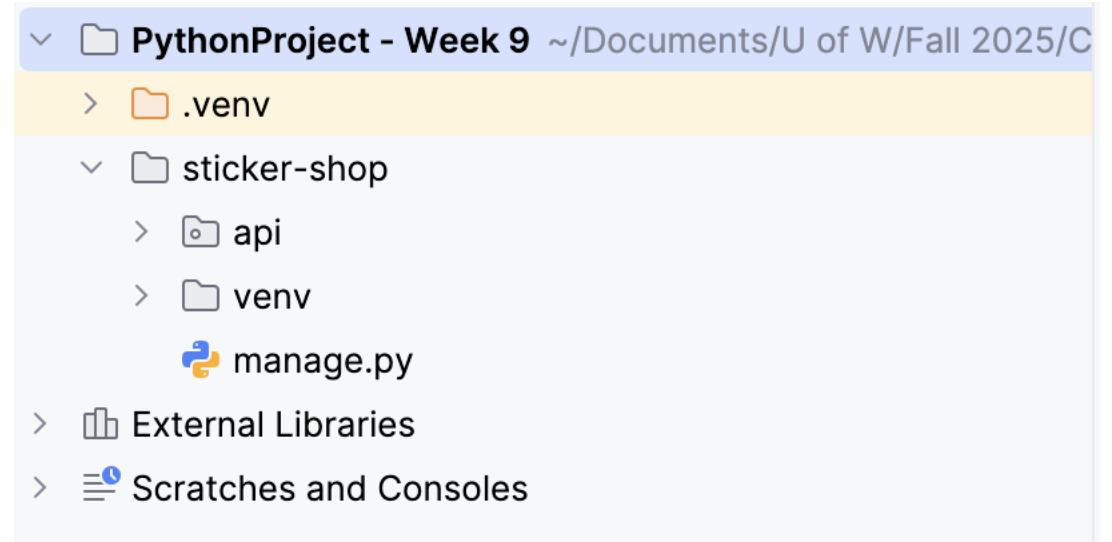
```
pip install django djangorestframework celery redis
```

We will add RabbitMQ and Kafka later – first we get the core working.

Step 4 – Create the Django Project

```
django-admin startproject api .
```

You now have:



Project: “Sticker Shop”

Step 5 – Create an App for Orders

#MacOS

```
python3 manage.py startapp orders
```

#Windows

```
python manage.py startapp orders
```

Step 6 – Enable the App & DRF(Django REST Framework) in settings

Open `api/settings.py` and find `INSTALLED_APPS`.

Add:

```
INSTALLED_APPS = [  
    ...  
    'rest_framework',  
    'orders',  
]
```

Project: “Sticker Shop”

Step 7 – Create the Order Model

Edit `orders/models.py`:

```
from django.db import models
import uuid

class Order(models.Model):
    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
    email = models.EmailField()
    item = models.CharField(max_length=120)
    quantity = models.PositiveIntegerField(default=1)
    total_cents = models.PositiveIntegerField(default=0)
    idempotency_key = models.CharField(max_length=64, unique=True)
    created_at = models.DateTimeField(auto_now_add=True)
```

- By default, Django creates an auto-incrementing integer ID.
- Here, we replaced it with a **UUID** (a globally unique identifier).
- `uuid.uuid4` generates a random UUID, e.g.:
a8c8b9a1-4d83-4138-8f48-21f441c1ef90

Project: “Sticker Shop”

Step 8 – Create DB Tables

#MacOS

```
python3 manage.py makemigrations
python3 manage.py migrate
```

#Windows

```
python manage.py makemigrations
python manage.py migrate
```

Step 9 – Create a Serializer and View

Create orders/serializers.py:

```
from rest_framework import serializers
from .models import Order

class OrderCreateSerializer(serializers.ModelSerializer):
    class Meta:
        model = Order
        fields = ['email', 'item', 'quantity', 'idempotency_key']

    def validate(self, data):
        data['item'] = data['item'].strip()
        data['quantity'] = max(1, data['quantity'])
        data['total_cents'] = data['quantity'] * 299
        return data
```

Project: “Sticker Shop”

Step 10 – Create orders/views.py:

```
from rest_framework.decorators import api_view
from rest_framework.response import Response
from rest_framework import status
from django.db import transaction, IntegrityError
from .serializers import OrderCreateSerializer
from .models import Order

@api_view(["POST"])
def create_order(request):
    ser = OrderCreateSerializer(data=request.data)
    ser.is_valid(raise_exception=True)

    try:
        with transaction.atomic():
            order = Order.objects.create(**ser.validated_data)
    except IntegrityError:
        order =
Order.objects.get(idempotency_key=ser.validated_data["idempotency_key"])

    return Response({"order_id": str(order.id)}, status=status.HTTP_201_CREATED)
```

Project: “Sticker Shop”

Step 11 – Register URL in orders/urls.py

```
from django.urls import path
from .views import create_order

urlpatterns = [ path('', create_order, name='create_order') ]
```

api/urls.py

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('orders/', include('orders.urls')),
]
```

Project: “Sticker Shop”

Step 12 – Test the API Before Adding Queues

Start the server:

```
python manage.py runserver
```

Send a test request (use browser, curl, or Postman):

```
curl -X POST http://127.0.0.1:8000/orders/ \
-H "Content-Type: application/json" \
-d '{"email":"test@example.com","item":"Cat
Sticker","quantity":2,"idempotency_key":"abc123"}'
```

```
(.venv) os@Olana-Syrotkina PythonProject - Week 9 % curl -X POST http://127.0.0.1:8000/orders/ \
-H "Content-Type: application/json" \
-d '{"email":"test@example.com","item":"Cat Sticker","quantity":2,"idempotency_key":"abc123"}'
{"order_id":"d812408b-dc51-42bd-8777-d317895221c7"}%
```

```
(.venv) os@Olana-Syrotkina PythonProject - Week 9 %
```

Your API is now working!

Project: “Sticker Shop”

Step 12 – Add RabbitMQ + Celery (Background Jobs)

Component	Purpose
RabbitMQ	Message Queue to offload work
Celery Worker	Background worker that processes jobs
Redis	(Optional) stores Celery results

Add RabbitMQ (via Docker)

docker-compose.yml at the project root (next to **manage.py**). If you already have one, add this service:

```
version: "3.9"
services:
  rabbitmq:
    image: rabbitmq:3.13-management
    container_name: classroom_rabbitmq
    ports:
      - "5672:5672"          # AMQP
      - "15672:15672"       # Management UI
    environment:
      RABBITMQ_DEFAULT_USER: guest
      RABBITMQ_DEFAULT_PASS: guest
```

Project: “Sticker Shop”

Step 12 – Install Docker Desktop

<https://www.docker.com/products/docker-desktop/>

Run:

```
docker ps  
cd sticker-shop
```

macOS / Linux/ Windows (PowerShell)

```
docker compose up -d rabbitmq
```

Open the RabbitMQ UI at <http://localhost:15672> (user **guest**, pass **guest**) to confirm it's running.

Project: “Sticker Shop”

Open the RabbitMQ UI at <http://localhost:15672> (user **guest**, pass **guest**) to confirm it’s running.



RabbitMQ 3.13.7 Erlang 26.2.5.16

Refreshed 2025-11-11 01:00:04 Refresh every 5 seconds Virtual host All Cluster rabbit@74a07d63b8ac User guest Log out

OverviewConnectionsChannelsExchangesQueues and StreamsAdmin

Overview

Totals

Queued messages last minute ?

Currently idle

Message rates last minute ?

Currently idle

Global counts ?

Connections: 0Channels: 0Exchanges: 7Queues: 0Consumers: 0

Nodes

Name	File descriptors ?	Socket descriptors ?	Erlang processes	Memory ?	Disk space	Uptime	Cores	Info	Reset stats	+/-
rabbit@74a07d63b8ac	44 1048576 available	0 943629 available	447 1048576 available	167 MiB 3.1 GiB high watermark	428 GiB 48 MiB low watermark	2m 32s	10	basic 2 rss	This node All nodes	

Project: “Sticker Shop”

Step 13 – Install Celery in your Django environment

From your *virtual environment* for this project:

macOS

```
pip install celery==5.4.0  
pip freeze | grep celery
```

Windows (PowerShell)

```
pip install celery==5.4.0  
pip show celery
```

Step 14 – Configure Celery for the Django project

Create Celery app for Django

Create `api/celery.py`

Project: “Sticker Shop”

Create Celery app for Django

Create **api/celery.py**

```
# api/celery.py
import os
from celery import Celery

# Tell Celery which Django settings to use
os.environ.setdefault("DJANGO_SETTINGS_MODULE",
"api.settings")

app = Celery("api")

# Use environment variable if provided; default to local RabbitMQ
app.conf.broker_url =
os.environ.get("CELERY_BROKER_URL",
"amqp://guest:guest@localhost:5672//")
```

```
# Backend
app.conf.result_backend =
os.environ.get("CELERY_RESULT_BACKEND",
"rpc://")

# Make Celery read settings with CELERY_ prefix
from Django settings.py (optional)
app.config_from_object("django.conf:settings",
namespace="CELERY")

# Auto-discover tasks.py in all INSTALLED_APPS
(e.g., orders/tasks.py)
app.autodiscover_tasks()

# Optional: set defaults
app.conf.timezone = "UTC"
app.conf.task_track_started = True
app.conf.task_time_limit = 30 * 60 # 30 minutes
```

Project: “Sticker Shop”

Edit api/init.py (so celery.py loads when Django starts)

```
# api/__init__.py
from .celery import app as celery_app

__all__ = ("celery_app",)
```

Add a Celery task in your app

```
# orders/tasks.py
from celery import shared_task
from time import sleep

@shared_task(bind=True)
def process_order(self, order_id: int):
    """
    Demo task: pretend to process an order.
    Retries automatically if something transient goes wrong.
    """
    # Simulate work
    sleep(2)
    return {"status": "processed", "order_id": order_id}
```

Project: “Sticker Shop”

Call the task from your view

`orders/views.py`

```
from rest_framework.decorators import api_view
from rest_framework.response import Response
from uuid import uuid4
# from .tasks import process_order    # uncomment after Celery is wired

@api_view(["GET", "POST"])
def create_order(request):
    # If GET, make a demo order so you can test quickly from the browser.
    if request.method == "GET":
        order = {"id": str(uuid4()), "items": ["demo-widget"], "qty": 1}
    else:    # POST
        order = request.data or {"id": str(uuid4())}

    # Kick off async work (uncomment once Celery is set up)
    # task = process_order.delay(order)

    # For now, return a queued-style response
    return Response({
        "message": "queued",
        # "task_id": task.id,           # add back after Celery
        "order": order
    }, status=202)
```

Project: “Sticker Shop”

api/urls.py

```
path('orders2/', include('orders.urls')),
```

orders/urls.py

```
urlpatterns = [ path('', create_order, name='create_order'),  
                path('', create_order2, name='create_order2'),  
                ]
```

Configure Django settings

api/settings.py

```
# Celery settings via namespace  
CELERY_BROKER_URL = "amqp://guest:guest@localhost:5672/"  
CELERY_RESULT_BACKEND = "rpc://"   
CELERY_TASK_ALWAYS_EAGER = False # keep False; True runs tasks inline (for tests)  
CELERY_TASK_TIME_LIMIT = 1800
```

Project: “Sticker Shop”

Run everything

1) Start RabbitMQ (if using Docker)

docker compose up -d rabbitmq

2) Run Django

python manage.py runserver

3) Start a Celery worker (new terminal)

celery -A api worker -l info

You might need to change your directory for that

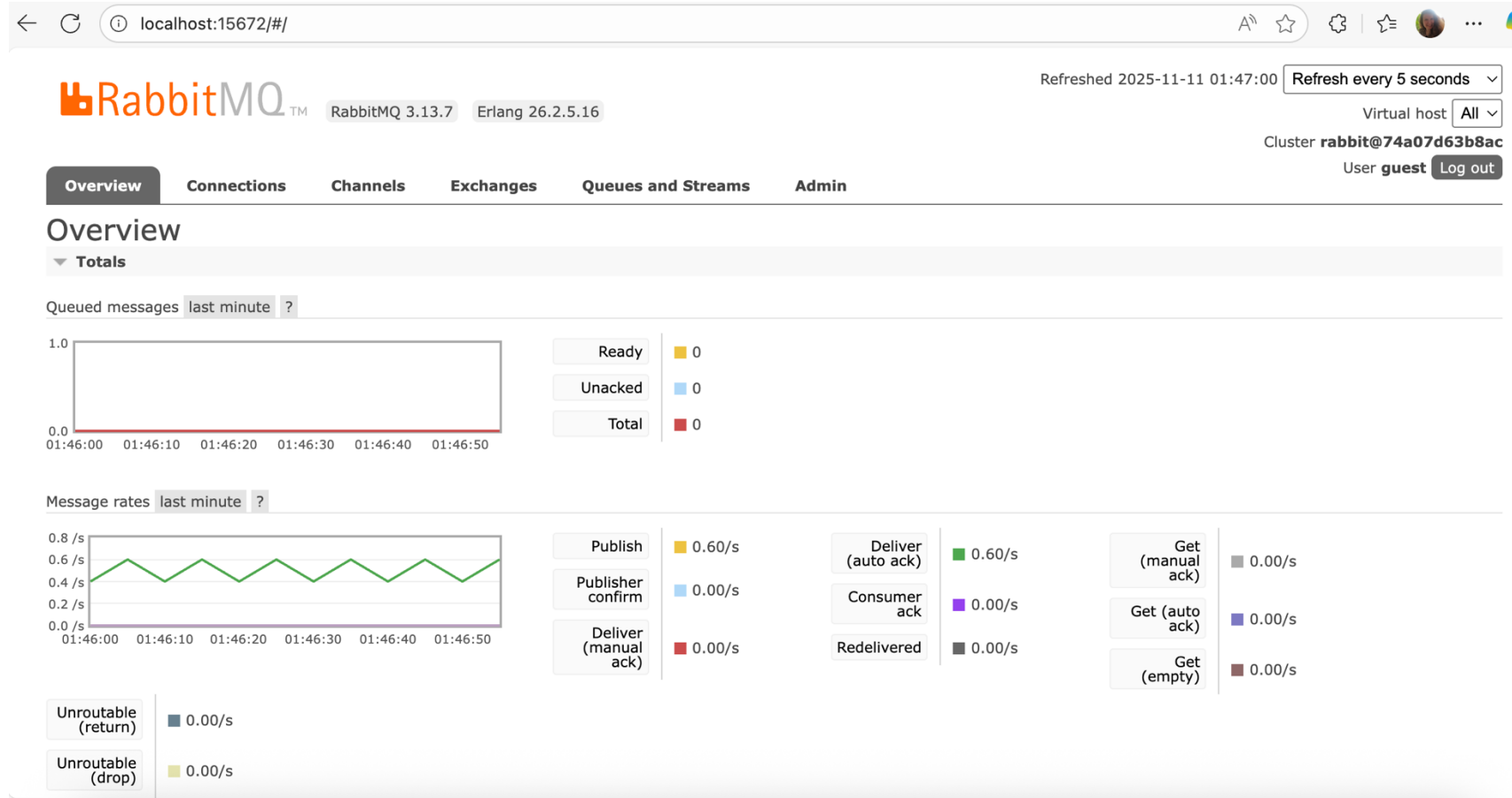
cd sticker-shop

Visit:

- <http://127.0.0.1:8000/orders2/> → should return {"message":"queued", ...}
- RabbitMQ UI: <http://localhost:15672> (guest/guest) → see connections, queues, messages.

Project: “Sticker Shop”

RabbitMQ UI: <http://localhost:15672> (guest/guest) → see connections, queues, messages.



Project: “Sticker Shop”

Create a queue

1. Click Queues and Streams (top menu).
2. Click Add a new queue.

Fill in:

Field	Value
Name	demo_queue
Durability	leave default
The rest	leave as is

Overview		Connections	Channels	Exchanges	Queues and Streams			Admin					
Overview								Messages			Message rates		
Virtual host	Name	Type	Features			State	Ready	Unacked	Total	incoming	deliver / get	ack	
/	celery	classic	D			running	0	0	0				
/	celery@Olena-Syrotkina.local.celery.pidbox	classic	AD	TTL	Exp	running	0	0	0				
/	celeryev.d10da077-7067-4204-a93c-4a8a3908d625	classic	AD	TTL	Exp	running	0	0	0	0.60/s	0.60/s	0.00/s	
/	demo_queue	classic	D			running	0	0	0				

Put messages (fake data) into the queue

1. Click on demo_queue in the list.
2. Scroll a little down to the box **Publish message**.
3. In the **Payload** box, type anything, like: Hello 1, Hello 2 and Hello 3.

Overview						Messages			Message rates					
Virtual host	Name	Type	Features			State	Ready	Unacked	Total	incoming	deliver / get	ack		
/	celery	classic	D			running	0	0	0					
/	celery@Olena-Syrotkina.local.celery.pidbox	classic	AD	TTL	Exp	running	0	0	0					
/	celeryev.d10da077-7067-4204-a93c-4a8a3908d625	classic	AD	TTL	Exp	running	0	0	0	0.40/s	0.40/s	0.00/s		
/	demo_queue	classic	D			running	3	0	3	0.00/s				

▼ Add a new queue